BUILDSITE

Construction Operations Management System

OBJECT-ORIENTED PROGRAMMING
MINI PROJECT REPORT

Submitted by
Aayan Khan — R036
Prathmesh Asavle — R006
Aarnav Kaaza — R032

MBA Tech – Artificial Intelligence
Mukesh Patel School of Technology Management & Engineering
NMIMS University

Academic Year 2026–2027

Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement
- 4. Aim and Objectives
- 5. Scope of the Project
- 6. Technology Stack
- 7. System Architecture
- 8. Module Description
- 9. Database and Data Models
- 10. Object-Oriented Programming Concepts
- 11. Validation and Error Handling
- 12. Testing and Verification
- 13. Limitations
- 14. Future Enhancements
- 15. Conclusion
- 16. References

1. Abstract

BuildSite is a Java-based web application designed to support essential construction-site operations from a single interface. The current Phase 1 implementation provides a dashboard, project management, task management, inventory management, and an interactive page demonstrating core Object-Oriented Programming (OOP) principles. The application is built with Spring Boot, Spring MVC, Thymeleaf, Spring Data JPA, Hibernate, and an H2 database. It uses server-rendered web pages and database-backed records to present operational information and process user-submitted forms. Inventory validation prevents stock from becoming negative and prevents issuing more material than is available. The project demonstrates how Java OOP concepts can be applied within a practical web application, while leaving authentication, audit history, and several additional site-management modules as future work.

2. Introduction

Construction projects require coordination between project planning, work assignments, and the materials available at a site. When information is tracked in separate notes or spreadsheets, it can be difficult to maintain a consistent view of current project progress, pending tasks, and material stock. BuildSite addresses this problem at a prototype level by bringing selected operations into one web-based application.

The project is also an academic demonstration of Java application development. It connects a browser interface to Java controllers, domain models, repositories, and a relational database. This structure makes it possible to demonstrate object modelling, encapsulation, inheritance, abstraction, interfaces, and runtime polymorphism in a recognizable real-world context.

3. Problem Statement

Small construction teams may rely on disconnected tools to track projects, work items, and materials. This can create duplicated data, limited visibility, and avoidable stock-handling mistakes. The project problem is to develop a simple central application that can record and display core construction operations and enforce basic rules when users update records.

4. Aim and Objectives

Aim: To design and implement a Java web application prototype for managing selected construction-site operations.

- Provide a dashboard with operational metrics derived from stored records.
- Allow users to create, view, edit, and delete projects.
- Allow tasks to be created and associated with a project.
- Maintain material records and support receiving and issuing stock.
- Validate stock operations so that stock does not become negative.
- Demonstrate OOP principles using Java classes, inheritance, abstract classes, interfaces, and method overriding.
- Use a layered web architecture with controllers, models, repositories, and a service component.

5. Scope of the Project

The report describes the Phase 1 source package supplied for review. The implemented scope includes the dashboard, project CRUD (create, read, update, delete), task listing and creation with project assignment, inventory/material operations, and an OOP demonstration page.

Navigation items for additional modules may appear in the interface, but the supplied README identifies expenses, equipment maintenance, attendance, site diary, formal stock transaction history, CSV reports, login, permissions, and audit logging as planned later-phase work. They are therefore not described here as completed features.

6. Technology Stack

Technology	Role in BuildSite
Java	Main programming language for application logic and OOP models.
Spring Boot	Application setup and embedded web-server configuration.
Spring MVC / Spring Web	Maps HTTP requests to controller methods and returns views or redirects.
Thymeleaf	Renders dynamic HTML templates using server-side model data.
Spring Data JPA	Provides repository abstractions for database operations.
Hibernate	JPA implementation used to map Java entities to database tables.
H2 Database	Local relational database used for project, task, and material records.
HTML and CSS	Page structure and responsive visual styling.
Maven	Dependency management and project build configuration.

The supplied project configuration uses Spring Boot 3.4.4 and Java 21. The README states that a JDK 17 or newer is required. The current H2 configuration stores data locally in a file, so it can persist across application restarts when launched with the configured working directory.

7. System Architecture

BuildSite follows a conventional server-rendered Spring web application flow:

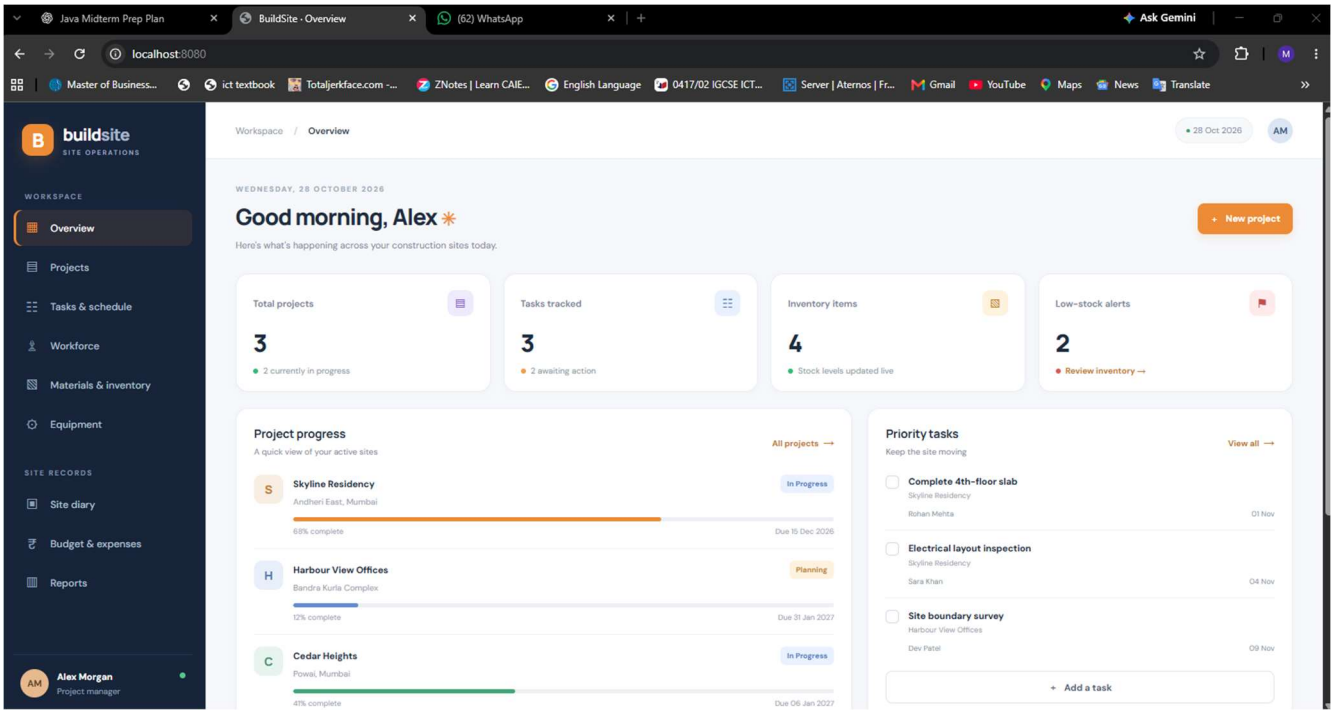
Layer	Responsibility
1. Browser / UI	The user interacts with HTML pages styled with CSS.
2. Thymeleaf templates	Templates display data supplied by controllers and render forms and tables.
3. Spring MVC controllers	Controllers handle incoming requests, validate form input, call repositories/services, and choose a view or redirect.
4. Service layer	DashboardService aggregates data used by the dashboard. Not every feature requires a separate service class in this phase.
5. Repositories and JPA	Spring Data JPA repositories provide database access for the domain models.
6. H2 database	Records are stored in local relational tables and retrieved when pages are rendered.

Typical request flow: a user submits a form in the browser; Spring MVC routes the request to the relevant controller; the controller checks the input and uses a repository or service; JPA/Hibernate interacts with H2; the controller then redirects or renders a page showing the updated information.

8. Module Description

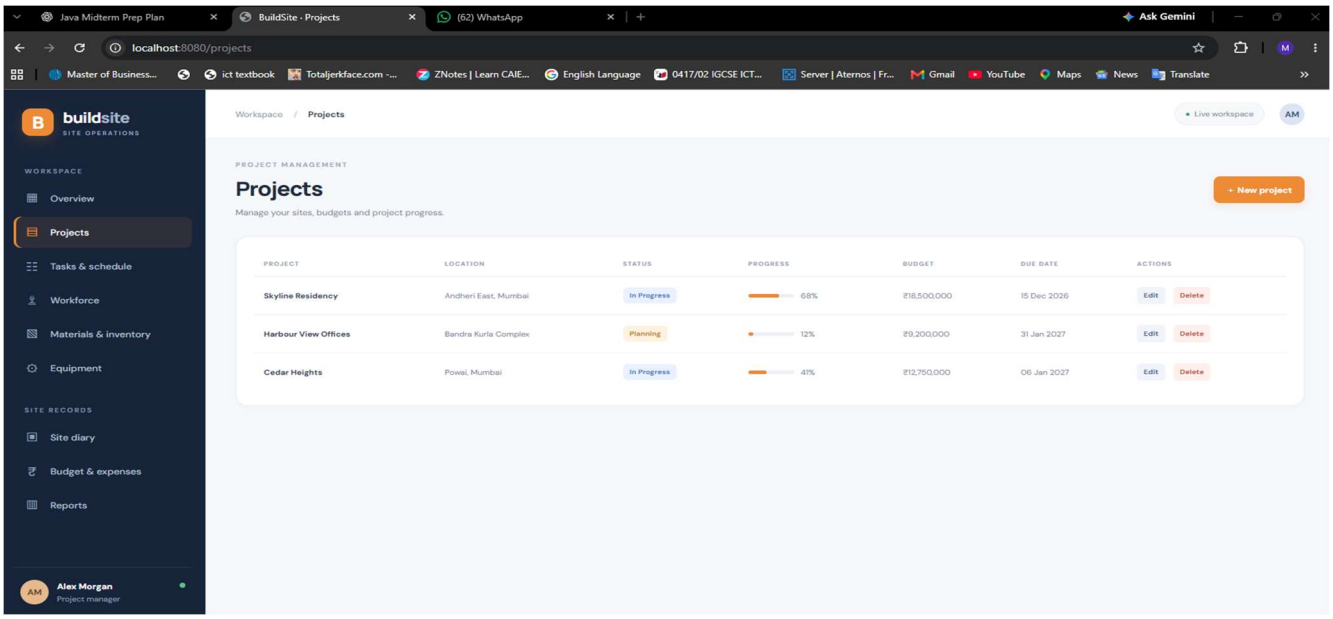
8.1 Dashboard

The dashboard provides a summary of available operational data. The DashboardController and DashboardService work together to retrieve and calculate metrics from database records, which are then displayed in the dashboard template.



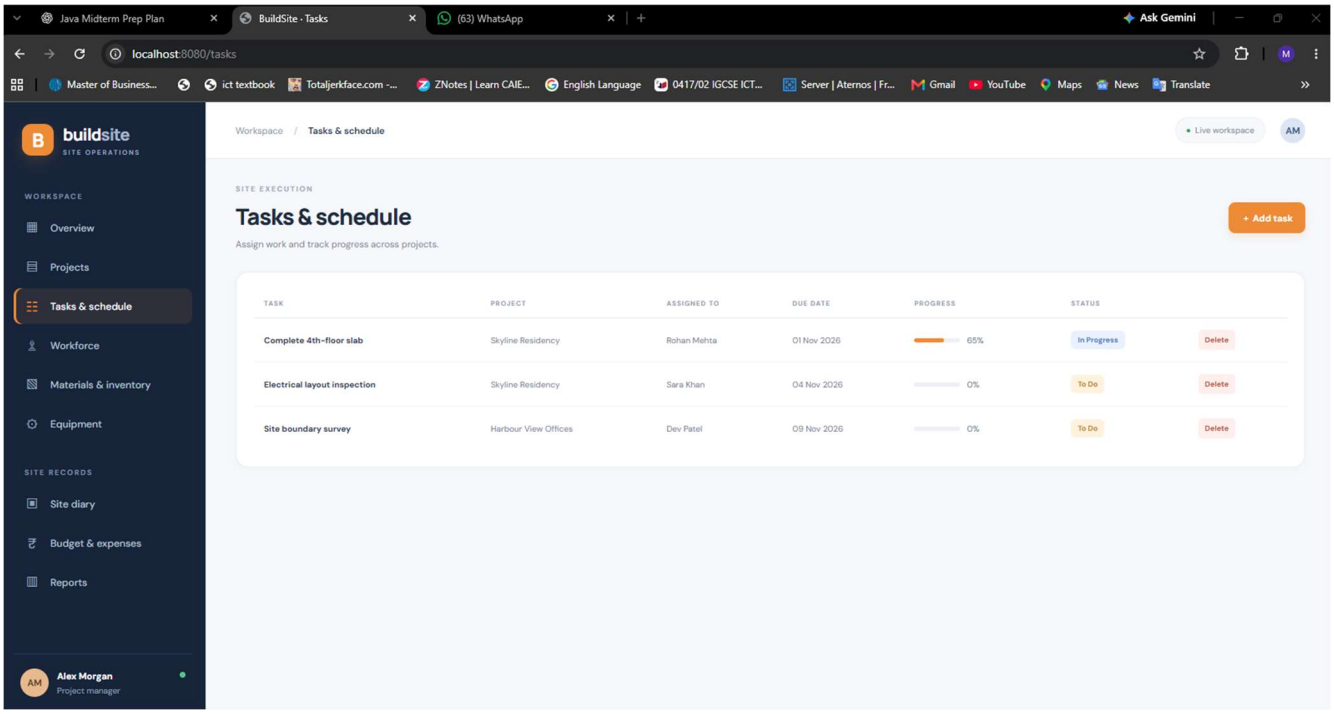
8.2 Project Management

The project module supports viewing the project list and creating, editing, and deleting project records. The Project model stores project attributes such as its name, location, status, progress, budget, and due date. The model includes an updateProgress method that checks that progress remains within the permitted range.



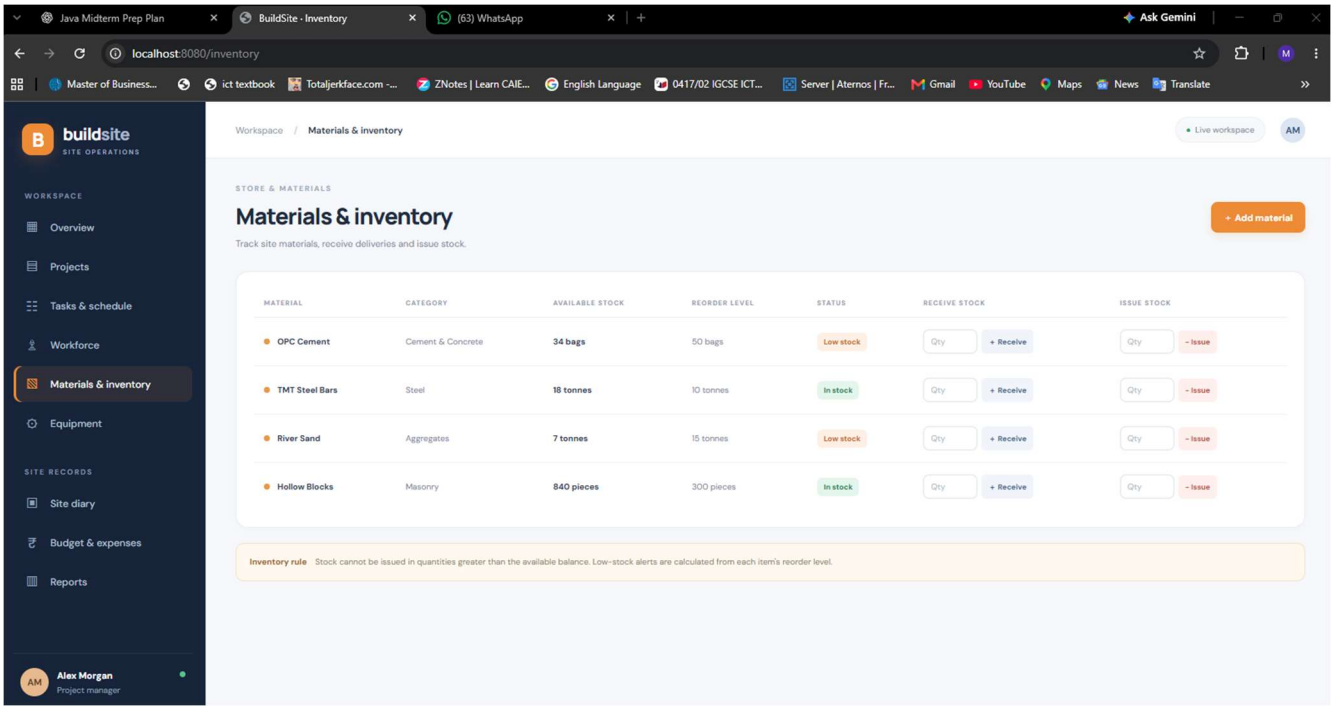
8.3 Task Management

The task module supports displaying tasks and creating new tasks associated with a project. This links an individual work item to the project it belongs to and provides a basic foundation for tracking site work.



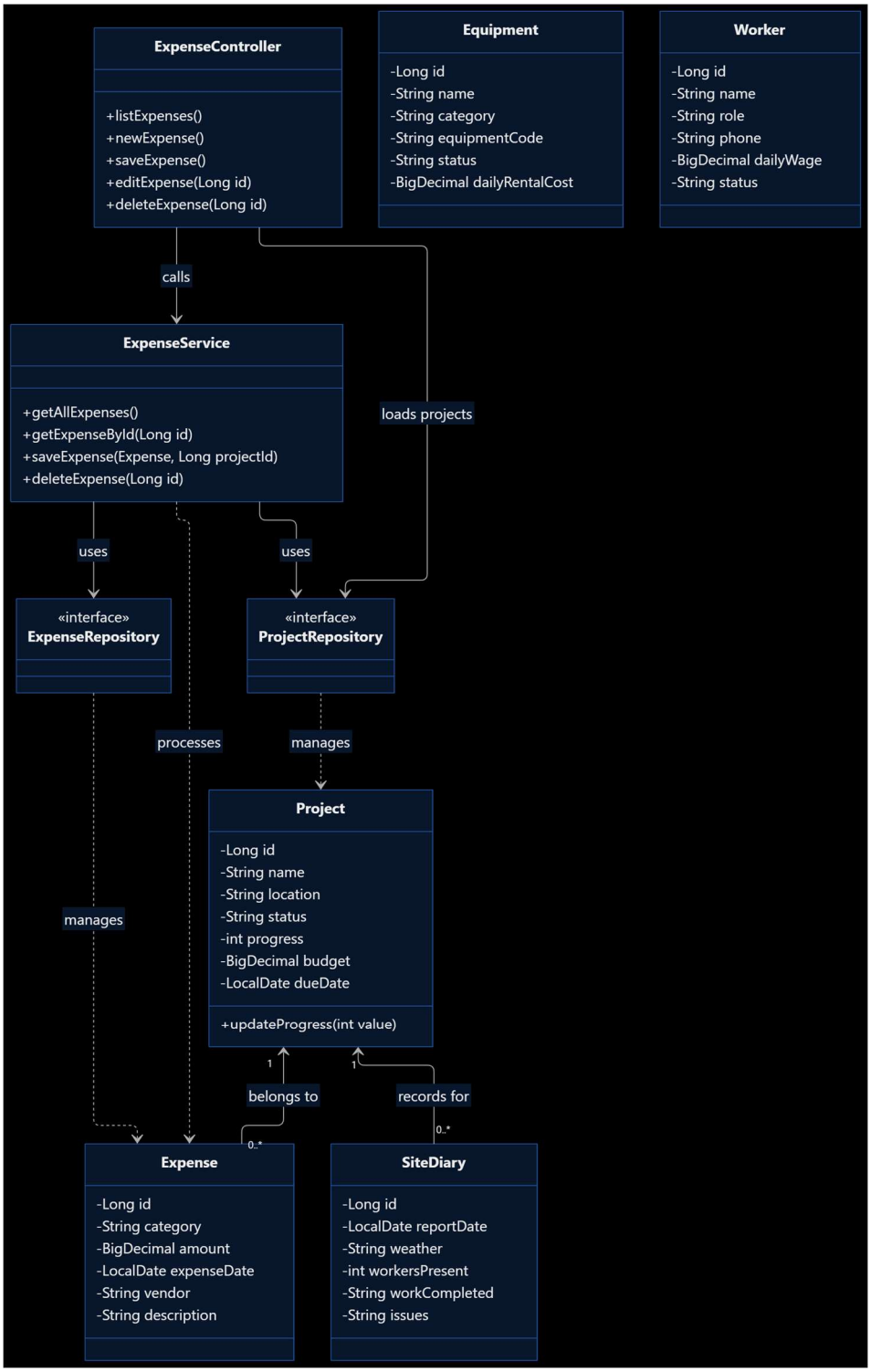
8.4 Inventory / Materials

The inventory module lists materials and supports adding material records, receiving stock, and issuing stock. The controller checks quantities before completing stock changes. Issuing a quantity greater than the available balance is rejected, and the application is designed to prevent negative stock.



8.5 OOP Demonstration

The OOP demonstration page uses a User hierarchy with Admin, SiteEngineer, and Storekeeper subclasses. It presents role-specific behavior and uses a collection of User references to demonstrate runtime polymorphism. Interfaces such as Approvable, Assignable, Notifiable, and Reportable are included in the OOP package to represent behavior contracts.



9. Database and Data Models

Model / Component	Purpose
Project	Represents a construction project and its key descriptive and planning fields.
TaskItem	Represents a task and its association with a project.
Material	Represents an inventory material and its stock quantity.
ProjectRepository	Provides data access operations for Project records.
TaskRepository	Provides data access operations for TaskItem records.
MaterialRepository	Provides data access operations for Material records.
DataSeeder	Adds starter sample records when the relevant tables are empty.

The application uses JPA entity mapping and repository interfaces to reduce the amount of low-level SQL required for routine data access. The starter records make it possible to inspect the dashboard and lists immediately after the first run. The supplied README indicates that the database is file-based in this phase; the H2 console is intended for local development and should not be exposed publicly.

10. Object-Oriented Programming Concepts

10.1 Encapsulation

Encapsulation keeps an object's state controlled by its class. The domain models use private fields and expose methods to read or update values. For example, Project.updateProgress() validates the progress value rather than allowing an unchecked update.

10.2 Inheritance

Admin, SiteEngineer, and Storekeeper extend the abstract User class. They reuse common user-related structure while representing different roles in the construction operations scenario.

10.3 Abstraction

User is abstract and defines the getRole() behavior that subclasses must provide. This captures common characteristics in a parent type while leaving role-specific details to concrete subclasses.

10.4 Runtime Polymorphism

The OOP demo places different subclass instances in a List<User> and invokes getRole() through User references. Java dispatches the call to the overridden implementation belonging to each actual object.

10.5 Interfaces

The project includes Approvable, Assignable, Notifiable, and Reportable interfaces to express behavior contracts. An interface allows classes to share a capability contract without requiring them to inherit implementation from one common concrete class. Specific implementation claims should be tied to the classes that implement each interface in the source code.

10.6 Exception-Based Validation

The project uses validation checks to reject invalid operations. For example, project progress is constrained to a valid range, and inventory operations reject quantities that are non-positive or exceed the available stock. This protects the consistency of application data.

11. Validation and Error Handling

- Project progress updates are checked against the allowed range.
- Inventory issue quantities are checked before stock is reduced.
- The application prevents issuing more material than the current available balance.
- Form fields use input constraints where appropriate, with server-side checks handling important business rules.
- Starter data is inserted only when the relevant tables are empty, avoiding repeated seeding on every page request.

Client-side constraints improve the form experience but are not a substitute for server-side validation. The stock checks in the controller are particularly important because browser-side validation can be bypassed.

12. Testing and Verification

The supplied project includes a ProjectTest test source. The following manual checks are recommended for a classroom demonstration; they describe verification steps rather than claiming that every test has already been executed.

Test	Action	Expected result
Application startup	Start the Spring Boot application and open the local URL.	Dashboard loads without a startup error.
Project creation	Create a project with valid values.	New project appears in the project list.
Project edit/delete	Update a project, then delete a test record.	The list reflects the changes.
Task creation	Create a task and assign it to a project.	Task appears in the task list with its project association.
Receive stock	Increase a material's stock quantity.	Displayed quantity increases.
Issue valid stock	Issue a positive quantity not exceeding stock.	Displayed quantity decreases.
Issue excess stock	Attempt to issue more than available.	Operation is rejected; stock does not become negative.
OOP demo	Open the OOP page and inspect the role examples.	The role hierarchy and overridden behavior are shown.

13. Limitations

- The current package is a Phase 1 academic prototype and is not production-secured.
- Login and role-based authorization are not implemented; the OOP role classes are demonstrations, not real user accounts.
- Audit logging and formal stock transaction history are not included in this phase.
- Attendance, expenses, equipment maintenance, site diary, and CSV reports are planned future modules in the supplied README.
- The current database setup is intended for local development. Public deployment would require security hardening and careful database configuration.

14. Future Enhancements

- Add secure authentication and authorization with role-based access control.
- Introduce audit logs and a formal stock movement ledger.
- Develop attendance, expense tracking, equipment maintenance, and site diary modules.

- Add report exports, filtering, search, and date-based analytics.
- Add stronger automated tests for project validation, task creation, and inventory edge cases.
- Deploy using a production database and environment-based configuration; disable public access to the H2 console.
- Improve accessibility, user feedback, and responsive behavior across mobile and desktop screens.

15. Conclusion

BuildSite demonstrates how Java and the Spring ecosystem can be used to create a web application for selected construction-site operations. Its project, task, and material modules provide a practical context for database-backed CRUD operations and business-rule validation. The OOP demonstration connects abstract programming concepts to role-based examples relevant to the domain. Although the supplied Phase 1 package is not a complete production management platform, it provides a structured foundation that can be expanded with security, additional operational modules, reporting, and more comprehensive testing.

16. References

- Project source package: BuildSite-Phase1-FIXED.zip (README.md, Java source files, Thymeleaf templates, application.properties, and test source).
- Spring Boot documentation: https://docs.spring.io/spring-boot/index.html
- Spring Framework documentation: https://docs.spring.io/spring-framework/reference/web/webmvc.html
- Thymeleaf documentation: https://www.thymeleaf.org/documentation.html
- Spring Data JPA documentation: https://docs.spring.io/spring-data/jpa/reference/
- H2 Database documentation: https://www.h2database.com/html/main.html
- The github Link: https://github.com/Burnoy/BuildSite-UML-Design

— End of Report —